음성 AI 전문가 과정 · Day 8 / 40

Day 08

추론 최적화 & Streaming ASR
더 빠르게 · 더 작게 · 말하는 도중에

병목 재기 → faster-whisper(CTranslate2) → 양자화 · compute_type → VAD · 배치 → whisper.cpp · q5_0 → 고정 청크 · LocalAgreement 스트리밍
2 주차 M2 · 4 시간 · Colab T4 권장 · 실습 노트북 Part 1 ~ 6

```
[ 같은 whisper-small, 같은 녹음 ]
transformers float32      RTF 0.30
faster-whisper float16    RTF 0.08
  + VAD + batch 8         RTF 0.03
whisper.cpp q5_0 (CPU)    181 MB

[ 스트리밍 · LocalAgreement]
1.0s   아이구루
2.0s   아이구루 교육센터에서 ▊
```

출력 모양 예시 · 숫자는 GPU 와 녹음에 따라 달라집니다

0.0 · 오늘 답할 질문

" 정확한 " Whisper 를 " 빠르고 작게 , 실시간으로 "

1	Whisper 는 어디서 시간을 쓰나 ? 무엇부터 고쳐야 하나 ?	Part 1
2	faster-whisper 는 같은 모델인데 왜 빠른가 ?	Part 2
3	양자화는 숫자를 어떻게 줄이고 , compute_type 은 무엇을 고를까 ?	Part 3 ~ 4
4	GPU 도 파이썬도 없는 기기에서는 어떻게 돌리나 ?	Part 5
5	말하는 도중에 자막을 띄우려면 ? 지연은 어떻게 재나 ?	Part 6

▶ 애니메이션 D8-1 오늘의 흐름

재고 → 엔진 바꾸고 → 줄이고 → 기기로 → 실시간으로

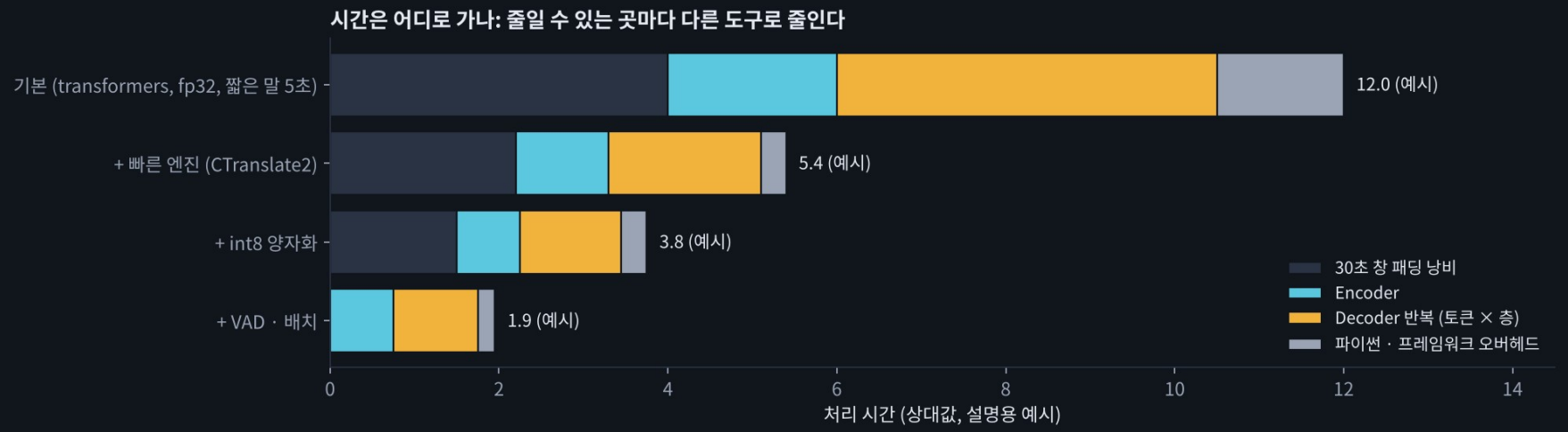
D8 · 같은 Whisper를 더 빠르고, 더 가볍고, 말하는 도중에



1~3교시: 녹음 파일을 빨리 처리하기 (배치 처리) → 4교시: 마이크 소리를 실시간으로 (스트리밍)

교시	내용	실습
1 교시	병목 재기 (Encoder vs Decoder) · faster-whisper · 게으른 생성기	Part 0 ~ 2 · 빈칸 1 ~ 2
2 교시	양자화 (int8 눈금) · compute_type · VAD · 배치	Part 3 ~ 4 · 빈칸 3 ~ 6
3 교시	whisper.cpp 빌드 · ggml 모델 · q5_0 양자화 · CLI	Part 5 · 빈칸 7
4 교시	지연 vs RTF · 고정 청크 · LocalAgreement · 지연 재기	Part 6 ~ 7 · 빈칸 8 ~ 9 · 노션

최적화의 첫걸음은 " 재는 것 "



시간이 새는 곳	오늘의 도구
30 초 창 패딩	VAD · 배치 (Part 4)
Decoder 반복	빠른 엔진 (Part 2) · turbo
숫자가 큼 (float32)	양자화 float16 · int8 (Part 3)
파이썬 오버헤드	C++ 엔진 : CTranslate2 · whisper.cpp

🍎 택배 배송 시간 줄이기

반쯤 빈 트럭 (패딩) 은 짐을 모아 싣고 (배치), 같은 동네 왕복 (Decoder) 은 경로를 줄이고 , 큰 짐 (float32) 은 압축 포장 (양자화), 느린 서류 (파이썬) 는 자동화 창구 (C++) 로 .

▶ 애니메이션 D8-2 병목 찾기 (최적화 켜기)

짧은 문장에서는 Decoder 가 시간을 더 쓴다

💬 **쉽게 말하면** Encoder 는 30 초를 " 한 번 " 듣고 , Decoder 는 글자 (토큰) 를 " 하나씩 반복 " 해서 적는다 .

```
t0 = time.time(); hf.model.encoder(f); enc_time = time.time() - t0
t0 = time.time(); ids = hf.generate(f, language="ko"); total = ...

dec_time = total - enc_time

# 전체 420 ms = Encoder 90 ms + Decoder 330 ms ( 토큰 21 개 )
```

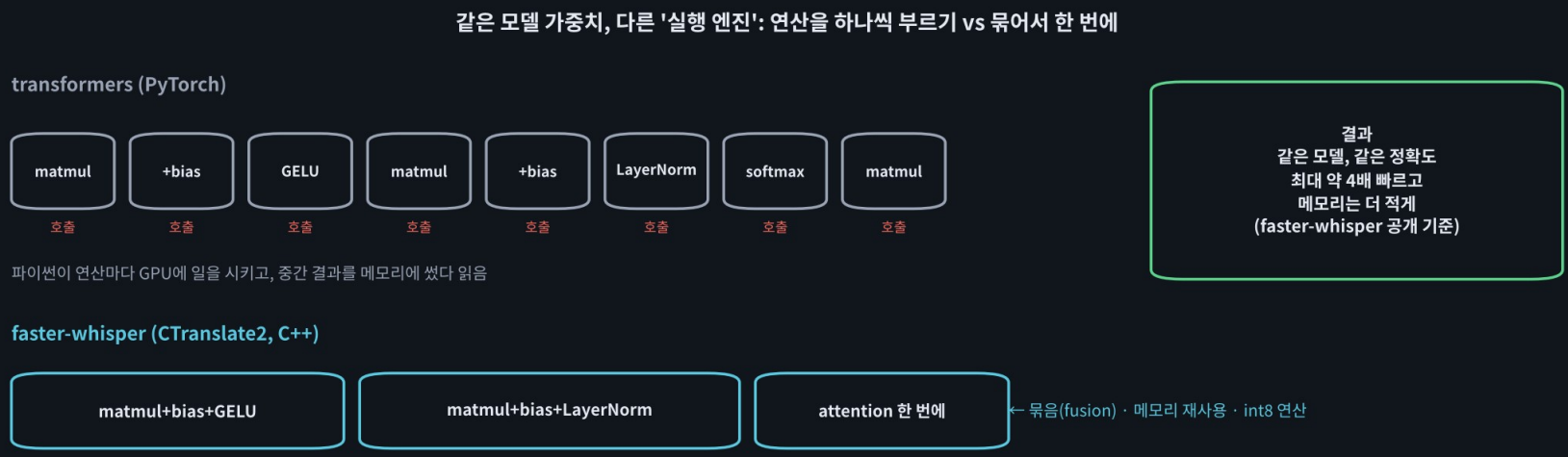
1. Encoder 시간만 따로 잰다
2. generate() 전체 시간을 잰다
3. 빼면 Decoder 몫
4. 토큰 수로 나누면 한 토큰당 시간

→ **반복 하나를 빠르게 = 엔진 교체**

🧩 **빈칸 실습 1**

dec_time = total - ____ 정답 : enc_time

같은 모델 , 더 빠른 엔진 (CTranslate2)



🍎 같은 악보 , 다른 연주자

악보 (가중치) 는 그대로인데 연주자 (엔진) 를 바꿨다 . 파이썬 연주자는 음표마다 지휘자를 쳐다보지만 , C++ 연주자는 여러 음을 한 번에 이어서 친다 (fusion).

	transformers	faster-whisper
가중치	같음	같음 (변환만)
엔진	PyTorch + 파이썬	CTranslate2 (C++)
기본 beam	1 (greedy)	5
결과	한 번에 문자열	게으른 생성기

▶ 애니메이션 D8-3 실행 엔진 (파이썬 vs C++)

transcribe() 는 " 주문서 " 만 준다 — 꺼낼 때 일한다

```
from faster_whisper import WhisperModel
CT = "float16" if DEVICE == "cuda" else "int8"
fw = WhisperModel("small", device=DEVICE, compute_type=CT)

segments, info = fw.transcribe(y, language="ko", beam_size=1)
segs = list(segments)      # ← 여기서 실제로 받아 적음
for s in segs:
    print(f"[{s.start:5.2f} → {s.end:5.2f}] {s.text}")
```

🍎 번호표 받은 식당

주문 (transcribe) 하면 번호표 (생성기) 만 받는다 .
음식은 번호를 불러 꺼낼 때 (for · list) 나온다 . 번호표 받은 시간을 " 요리 시간 " 이라 부르면 안 된다 .

기본 안전장치 (D7 과 같음)

compression_ratio 2.4 · log_prob −1.0 · no_speech 0.6 · temperature 0 → 1.0 사다리 · beam 5

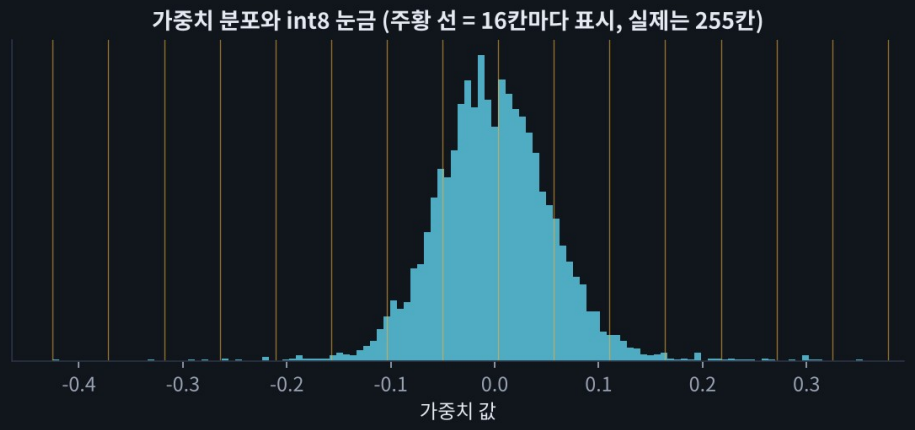
오늘 비교는 공정하게 beam_size=1

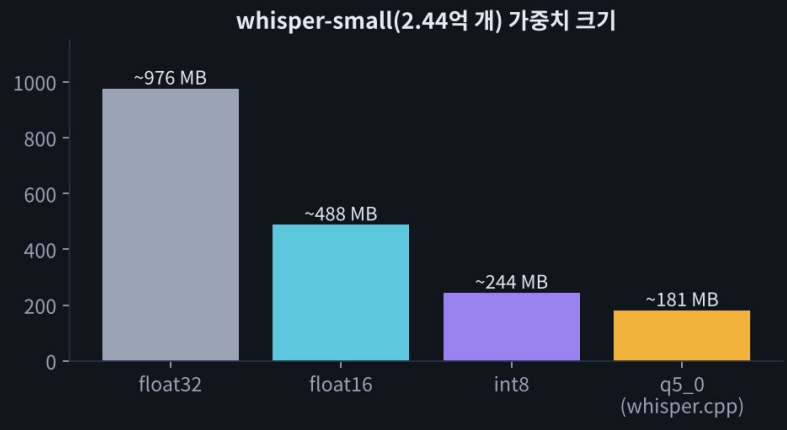
🧩 빈칸 실습 2

segs = ____(segments) 정답 : list

숫자를 작은 상자에 : float32 → float16 → int8

양자화 = 숫자를 더 적은 비트로 저장: 정확도는 조금 잃고, 크기 · 메모리 · 속도에서 크게 얻는다





📐 핵심 식 하나

`scale = max|w| ÷ 127,  q = round(w ÷ scale)`

읽는 법 : 가장 큰 값을 127 칸 자에 맞추고 , 각 숫자를 가장 가까운 눈금으로 . 되돌릴 땐 q × scale.

🧩 빈칸 실습 3

torch.round(w / ____) 정답 : scale

🧩 빈칸 실습 4

BYTES = {"int8": ____} 정답 : 1 → small 약 244 MB

▶ 애니메이션 D8-4 양자화 눈금 (비트 · 이상치 · 블록)

자 (ruler) 의 눈금을 줄이는 일

🍎 1 mm 자 → 1 cm 자

1 mm 눈금 자 (float32) 로 재던 것을 1 cm 눈금 자 (int8) 로 잰다 . 자가 가볍고 읽기 빠르지만 0.5 cm 까지 틀릴 수 있다 . 다행히 신경망은 이 정도 오차에 둔하다 .

1. 덩어리에서 가장 큰 절댓값을 찾는다
2. 그 값을 127 로 나눠 눈금 간격 (scale)
3. 모든 숫자를 scale 로 나눠 반올림 → int8
4. 계산할 때 q × scale 로 되돌린다

⚠ 이상치 (outlier) 하나가 망친다

값들이 대부분 ±0.1 인데 하나가 8.0 이면 , 눈금 간격이 8/127 ≈ 0.063 → 작은 값들이 0, ±1 로 뭉개진다 .

해결 : 블록 (32 개) 마다 scale 을 따로 → 이상치의 피해가 그 블록 안에 머문다 .

형식	숫자 하나	small 크기
float32	4 B	976 MB
float16	2 B	488 MB
int8	1 B	244 MB
q5_0 (ggml)	≈ 5.5 bit	≈ 181 MB

GPU 는 float16, CPU 는 int8 — 그래도 직접 재 보기

compute_type	어디서	특징
float32	CPU · GPU	가장 느림 , 정확도 비교의 기준점
float16	GPU	GPU 기본 , 정확도 거의 그대로
int8_float16	GPU	가중치 int8 + 계산 float16 → 메모리 절약
int8	CPU (GPU 도 가능)	CPU 에서 가장 빠름

```
for ct in ["float16", "int8_float16", "int8"]:
    m = WhisperModel("small", device="cuda", compute_type=ct)
    t, cer = run(m)       # RTF · CER · GPU 메모리를 한 표로
```

🍎 **포장 상자 크기 고르기**

작은 상자 (int8) 가 트럭에는 많이 실리지만 , 지게차 (GPU 텐서 코어) 가 중간 상자 (float16) 에 맞춰져 있으면 오히려 중간 상자가 빠르다 .

💬 **쉽게 말하면** " 작게 " 가 항상 " 빠르게 " 는 아니다 . 내 하드웨어에서 재 본 숫자가 정답이다 .

듣지 않아도 될 것은 빼고 , 한 번에 여러 개




```
segs, info = fw.transcribe(lecture, language="ko", vad_filter=True)
bp = BatchedInferencePipeline(model=fw)
segs, info = bp.transcribe(lecture, language="ko", batch_size=8)
```

🍎 **엘리베이터**

한 명씩 태우면 (창 하나씩) 느리다 . 여러 명을 한 번에 (배치) 태우면 빠르지만 , 정원 초과 (메모리 부족) 면 못 간다 .

🧩 **빈칸 실습 5**

vad_filter=___ → True

🧩 **빈칸 실습 6**

BATCH = ___ → 8

▶ 애니메이션 D8-5 VAD · 배치 (무음 · 엘리베이터)

파이썬 없이 , GPU 없이 — C++ 실행 파일 하나

whisper.cpp: 파이썬 · PyTorch 없이 C/C++ 실행 파일 하나로 (노트북 · 라즈베리파이 · 휴대폰 · Jetson)



입력은 16 kHz · 16-bit WAV · 같은 모델을 CPU(AVX), Apple(Metal), NVIDIA(CUDA), 휴대폰(ARM NEON)에서 그대로

→ D35 On-device / Edge에서 Jetson · sherpa-onnx와 함께 다시 씁니다

1. git clone whisper.cpp
2. cmake -B build → cmake --build build -j
3. download-ggml-model.sh small
4. whisper-quantize … q5_0
5. whisper-cli -m … -f a.wav -l ko

🍎 휴대용 통역기

통역 센터 (GPU 서버) 에 전화하지 않고 , 주머니 속 통역기 (기기 안 모델) 로 바로 통역 . 인터넷이 없어도 , 녹음이 밖으로 나가지 않아도 된다 .

▶ 애니메이션 D8-6 whisper.cpp 길잡이

q5_0: 크기는 약 40%, 정확도는 거의 그대로

```
!./whisper.cpp/build/bin/whisper-quantize \
    models/ggml-small.bin models/ggml-small-q5_0.bin q5_0

!./whisper.cpp/build/bin/whisper-cli \
    -m models/ggml-small-q5_0.bin -f story.wav \
    -l ko -t 4 -nt -otxt -of out
```

옵션	뜻
`-m`	모델 파일
`-f`	입력 WAV
`-l ko`	언어 (기본 en!)
`-t 4`	CPU 스레드
`-nt`	시각 표시 없이
`-otxt -of`	txt 로 저장

🧩 **빈칸 실습 7**

QTYPE = ____ 정답 : "q5_0"

모델	크기	쓰임
ggml-small.bin (f16)	≈ 466 MB	기준
ggml-small-q5_0.bin	≈ 181 MB	노트북 · 태블릿 · 라즈베리파이

상황에 맞는 도구 고르기

상황	추천	이유
GPU 서버에서 받아쓰기 API	faster-whisper float16 (+VAD)	빠르고 파이썬에서 쓰기 쉬움
회의 녹음 수백 개를 밤새 처리	BatchedInferencePipeline + VAD	처리량 최대
인터넷 없는 태블릿 · 현장	whisper.cpp + small/base q5_0	GPU · 파이썬 불필요 , 작음
모델 내부를 보며 배우기 · Fine-tuning	transformers	D6 · D7 · 3 주차
실시간 자막 · 음성 비서	faster-whisper + 스트리밍 (Part 6)	지연 1 ~ 2 초

💬 **쉽게 말하면** " 어디서 돌리나 (서버 · 기기)" 와 " 언제 결과가 필요한가 (나중 · 지금)" 부터 정하면 도구가 정해진다 .

사용자는 RTF 가 아니라 " 지연 " 을 느낀다

지표	질문	쓰는 곳
RTF	1 초 음성을 몇 초에 처리하나 ?	배치 처리 (Part 1 ~ 5)
지연 (latency)	말한 뒤 몇 초 만에 글자가 확정되나 ?	스트리밍 (Part 6)

🍎 동시통역 vs 번역 출판

번역 출판 (배치) 은 책을 다 받고 빨리 번역하면 된다 (RTF). 동시통역 (스트리밍) 은 말이 끝나기 전에 통역해야 하고 , 몇 초 늦게 따라가는지 (지연) 가 품질이다 .

Whisper 로 스트리밍하는 세 가지 요령

① 고정 청크 : N 초마다 자르기 (간단 , 단어 잘림)

② VAD 끝점 : 말이 끝나면 받아 적기 (음성 비서)

③ LocalAgreement: 버퍼를 계속 다시 받아 적고 두 번 같은 앞부분만 확정 (실시간 자막)

💬 **쉽게 말하면** " 빨리 계산 "(RTF) 은 필요조건 , " 빨리 보여 주기 "(지연) 가 사용자 경험이다 .

N 초마다 자르면 경계의 단어가 잘린다




```
CHUNK_N = int(3.0 * SR)
for i in range(0, len(story), CHUNK_N):
    piece = story[i : i + CHUNK_N]   # → 받아 적기
```

🍎 3 초마다 끊기는 전화

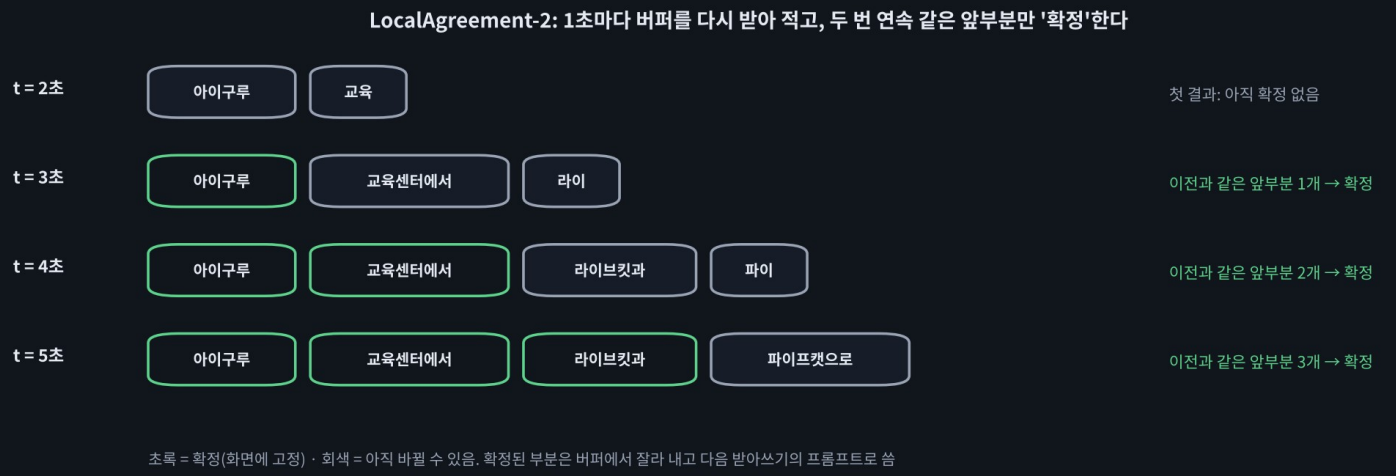
상대 말이 3 초마다 뚝뚝 끊기면 " 라이… " "… 브킷과 " 처럼 단어가 반쪽씩 들린다 .

🧩 빈칸 실습 8

range(0, len(story), ____) 정답 : CHUNK_N

▶ 애니메이션 D8-7 고정 청크 (길이 슬라이더)

두 번 연속 같은 앞부분만 확정한다



1. 새 소리 1 초를 버퍼에 붙인다
2. 버퍼 전체를 받아 적는다 (가설)
3. 지난 가설과 공통 앞부분 (lcp) 을 찾는다
4. 공통 부분만 확정 , 나머지는 다음 비교로

```
def lcp(a, b):
    i = 0
    while i < min(len(a), len(b)) and a[i] == b[i]:
        i += 1
    return i
```

🧩 **빈칸 실습 9**

a[i] == ____[i] 정답 : b

▶ 애니메이션 D8-8 LocalAgreement (한 걸음씩)

반쪽 단어를 성급하게 확정하지 않으려고

🍎 받아쓰기 시험의 두 친구

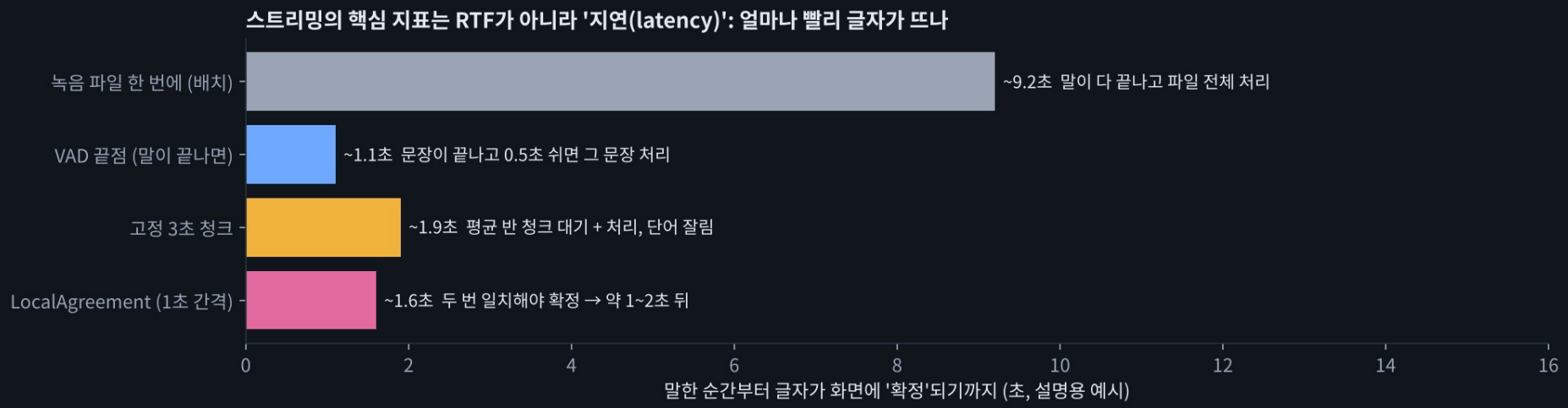
두 친구가 1초 간격으로 받아 적은 공책을 비교해, 둘 다 똑같이 적은 앞부분만 칠판에 옮겨 적는다. 한 명만 적은 반쪽 단어는 다음 비교까지 기다린다.

```
1.0s   가설:  아이구루 교육
2.0s   가설:  아이구루 교육센터에서 라이
       확정:  아이구루
3.0s   가설:  아이구루 교육센터에서 라이브킷과
       확정:  +  교육센터에서
끝     flush:  남은 단어 모두 확정
```

손잡이	효과
간격 0.5초	지연 ↓, 계산량 ↑
작은 모델 · int8 · GPU	계산 시간 ↓
확정 단어 → 프롬프트	문맥 이어 일관성 ↑
버퍼 잘라 내기	계산 시간 일정하게

💬 **쉽게 말하면** "두 번 들은 것만 믿는다." 대신 1~2초 늦다.

단어별 지연 = 확정된 시각 – 단어 끝 시각



1. word_timestamps=True 로 단어 끝 시각
2. 확정되는 순간의 시각을 기록
3. 빼면 단어별 지연 → 평균 · 최대
4. 매 걸음 계산 시간 < 간격 (1 초) 인지 확인

🍎 자막 방송

뉴스 자막이 화면보다 2 초 늦으면 참을 만하지만 10 초 늦으면 쓸모없다 . 그리고 점점 더 늦어지면 (계산 > 간격) 결국 무너진다 .

▶ 애니메이션 D8-9 지연 비교 (청크 · 끝점 · LA)

자주 틀리는 일곱 가지

헷갈림	바른 이해
transcribe() 시간 = 받아쓰기 시간 ?	아니다 . 생성기라 list() 까지 재야 한다
faster-whisper 는 다른 모델 ?	같은 가중치 , 다른 엔진 (CTranslate2)
int8 은 항상 float16 보다 빠르다 ?	하드웨어에 따라 다르다 . 직접 재기
vad_filter 를 켜면 시각이 바뀐다 ?	원래 녹음 기준 시각으로 되돌려 준다
배치를 키우면 지연이 준다 ?	배치는 처리량용 . 크면 메모리 부족
whisper-cli 는 언어 자동 ko?	기본 en. -l ko 필수
RTF < 1 이면 스트리밍 OK?	필요조건일 뿐 . 사용자는 지연을 느낀다

오늘 만든 것과 다음 단계

- ☐ Encoder · Decoder 시간을 재서 병목을 찾았다
- ☐ faster-whisper 로 같은 녹음을 더 빠르게 받아 적었다
- ☐ int8 양자화를 직접 계산하고 compute_type 을 비교했다
- ☐ VAD 와 배치로 긴 녹음을 빠르게 처리했다
- ☐ whisper.cpp 를 빌드하고 q5_0 모델로 받아 적었다
- ☐ 고정 청크와 LocalAgreement 스트리밍의 지연을 쟀다

다음 : Day 9 · WhisperX & 화자 분리

오늘 쓴 단어 시각 (word_timestamps) 은 추정이라 ± 수백 ms 흔들립니다 . D9 에서는 D6 의 CTC 정렬로 단어 시각을 정확히 맞추는 WhisperX, " 누가 말했나 " 를 나누는 pyannote 화자 분리를 배웁니다 . D10 회의록 생성기의 핵심 부품입니다 .

worksheet.html	40 문항
개념 워크시트 · 노션 제출	
animation.html	9 개 탭
인터랙티브 애니메이션	
D8_ 추론최적화 _ 스트리밍 _ 실습 .ipynb	빈칸 9
실습 노트북	

온라인 자료 imguru-mooc.github.io/AI_Voice/day08